NAME : AAKASH S ROLL NO: 231901001

EX NO. 2

DEVELOPING AND DEPLOYING A SIMPLE CHAINCODE

AIM

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a Hyperledger Fabric test network using the JavaScript programming language.

SOFTWARE REQUIREMENTS

- Ubuntu / Kali Linux
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Go (Version 1.20 or later)
- Git
- Curl
- jq
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images

HARDWARE REQUIREMENTS

- **Processor:** Intel Core i3 or above
- **RAM:** Minimum 8 GB
- **Storage:** Minimum 20 GB free disk space
- **Internet**

Connection PROCEDURE

PART A: ONE-TIME SYSTEM SETUP

Step 1: Update Ubuntu

Update the Ubuntu package repositories and upgrade the

installed packages. sudo apt update && sudo apt upgrade -y

sudo apt install -y curl git jq build-essential

Step 2: Install Docker

Install the required Docker dependencies.

Configure Docker Without sudo

Add the current user to the

Docker group. sudo usermod -aG

docker $USER newgrp docker

Verify Docker installation.

docker run hello-world

```
vboxuser@Ubuntu:~$ sudo usermod -aG docker $USER
newgrp docker
docker run hello-world   # should print "Hello from Docker!"
[sudo] password for vboxuser:
```

Step 3: Install Node.js and npm

Install Node.js Version 18 or later.

curl -fsSL https://deb.nodesource.com/setup_18.x |

sudo -E bash - sudo apt install -y nodejs

Verify

Node.js.

node -v

```
vboxuser@Ubuntu:~$ curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
node -v     # should show v18.x or later
npm -v
2026-08-28 18:13:35 -
================================================================================
                           DEPRECATION WARNING
================================================================================
Node.js 18.x is no longer actively supported!
You will not receive security or critical stability updates for this version.

You should migrate to a supported version of Node.js as soon as possible.
```

```
Removing libnode109:amd64 (18.19.1+dfsg-6ubuntu5) ...
(Reading database ... 199134 files and directories currently installed.)
Preparing to unpack .../nodejs_18.20.8-1nodesource1_amd64.deb ...
Unpacking nodejs (18.20.8-1nodesource1) over (18.19.1+dfsg-6ubuntu5) ...
(Reading database ... 204310 files and directories currently installed.)
Removing node-acorn (8.8.1+ds+~cs25.17.7-2) ...
Setting up nodejs (18.20.8-1nodesource1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
Processing triggers for man-db (2.12.0-4build2) ...
v18.20.8
10.8.2
```

Step 4: Install Go

Install Go required for building some Hyperledger Fabric

binaries. sudo apt install -y golang-go

Verify Go

installation. go

version

PART B: DOWNLOAD HYPERLEDGER FABRIC

Step 5: Create the Working Directory

Create a directory for the Hyperledger

Fabric project. mkdir -p ~/fabric-dev

cd ~/fabric-dev

Verify the current

directory. pwd

Step 6: Download Hyperledger Fabric Samples, Binaries and Docker Images

Download the Hyperledger Fabric installation script.

curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh

Give execute permission to the installation

script. chmod +x install-fabric.sh

Install the Fabric Docker images, samples, and binaries.

./install-fabric.sh docker samples

binary This downloads:

- fabric-samples/ directory
- Hyperledger Fabric sample applications
- peer CLI binary
- orderer binary
- configtxgen binary
- Hyperledger Fabric Docker images
- Peer, Orderer and Certificate Authority images

```
vboxuser@Ubuntu:~/fabric-dev$ curl -sSLO https://raw.githubusercontent.com/hyperledg
er/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

===> Changing directory to fabric-samples
fabric-samples v2.5.16 does not exist, defaulting to main. fabric-samples main branc
h is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.16 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.16/h
yperledger-fabric-linux-amd64-2.5.16.tar.gz
===> Will unpack to: /home/vboxuser/fabric-dev/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
```

Step 7: Configure Environment Variables

Add the Fabric binaries to the system PATH.

echo 'export PATH=$HOME/fabric-dev/fabric-samples/bin:$PATH' >>

~/.bashrc Configure the Fabric configuration path.

echo 'export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config' >>

~/.bashrc Reload the shell configuration.

source ~/.bashrc

Verify the Peer

CLI. peer version

Expected Output:

Version: v2.5.16

PART C: SET UP AND RUN THE FABRIC TEST NETWORK

Step 8: Navigate to the Test Network

Go to the Hyperledger Fabric test network

directory. cd ~/fabric-dev/fabric-

samples/test-network

Step 9: Start the Fabric Network with Certificate Authorities

Start the Fabric test network and create the application channel.

./network.sh up createChannel -ca

The command performs the following tasks:

- Starts the Orderer.
- Starts peer organizations.
- Starts Certificate Authorities.
- Generates cryptographic materials.
- Creates the application channel.
- Establishes the required Docker containers.

```
vboxuser@Ubuntu:~/fabric-dev$ cd ~/fabric-dev/fabric-samples/test-network
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChan
nel -ca
Using docker and docker compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of
'3' seconds and using database 'leveldb
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
WARN[0000] Found orphan containers (ca_orderer, ca_org1, ca_org2) for this project.
If you removed or renamed this service in your compose file, you can run this comman
d with the --remove-orphans flag to clean it up.
[+] up 3/3
 ✔ Container orderer.example.com    Started                                    2.3s
```

Step 10: Verify Running Docker Containers

Display the active Fabric

containers. docker ps

Expected containers include:

- orderer.example.com
- peer0.org1.example.com
- peer0.org2.example.com
- ca_org1
- ca_org2
- ca_orderer

PART D: DEVELOP AND DEPLOY SIMPLE CHAINCODE

Step 11: Deploy the JavaScript Asset Transfer Chaincode

Deploy the Asset Transfer Basic chaincode written in JavaScript.

./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript

The command performs the following chaincode lifecycle operations:

1. Packages the chaincode.
2. Installs the chaincode on the peers.
3. Approves the chaincode definition.
4. Commits the chaincode definition to the channel.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl
 javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
```

Step 12: Configure Peer CLI Environment Variables

Before executing the peer chaincode commands, configure the environment variables for Org1.

export PATH=${PWD}/../bin:$PATH

export FABRIC_CFG_PATH=${PWD}/../config/

```
vboxuser@Ubuntu:~/fabric-dev$ echo 'export PATH=$HOME/fabric-dev/fabric-samples/bin:
$PATH' >> ~/.bashrc
echo 'export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config' >> ~/.bashrc
source ~/.bashrc
peer version   # confirms peer CLI is reachable
peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="Org1MSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org
1.exa mple.com/peers/peer0.org1.example.com/tls/ca.crt

export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.
example
.com/users/Admin@org1.example.com/

msp export

CORE_PEER_ADDRESS=localhost:7051

export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orde
rers/orderer
.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

These variables configure the Peer CLI to communicate with the Org1 peer and Orderer securely.

```
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 2, Endorsement Plugin: escc, Validation Plugin: vscc, Approv
als: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export PATH=${PWD}/../bin:
$PATH
export FABRIC_CFG_PATH=${PWD}/../config/

# Set identity/context to Org1 peer
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.examp
le.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.c
om/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

export ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/ord
erer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$
```

Step 13: Verify Installed Chaincode

Check the chaincode installed on

the peer. peer lifecycle chaincode

queryinstalled

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode q
ueryinstalled

peer chaincode invoke -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls --cafile $ORDERER_CA \
  -C mychannel -n basic \
  -c '{"function":"InitLedger","Args":[]}'

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b6
29, Label: basic_1.0
2026-08-28 18:26:20.725 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chain
code invoke successful. result: status:200
[]
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$
```

Step 14: Initialize the Ledger

Invoke the InitLedger function to initialize the ledger with

sample assets. peer chaincode invoke -o localhost:7050 \

--ordererTLSHostnameOverride orderer.example.com \

--tls --cafile $ORDERER_CA \

-C mychannel -n basic \

-c '{"function":"InitLedger","Args":[]}'

Expected Output:

Chaincode invoke successful

The ledger is now initialized with sample asset records.

Step 15: Query All Assets

Retrieve all the assets stored in the

ledger. peer chaincode query -C

mychannel -n basic \

-c '{"Args":["GetAllAssets"]}'

Expected Output:

[

{

"ID": "asset1",

"Color": "blue",

"Size": 5,

"Owner": "Tomoko",
"AppraisedValue": 300
},
{
"ID": "asset2",
"Color": "red",
"Size": 5,
"Owner":
"Brad",
"AppraisedValue": 400
}
]

The output displays the asset records stored in the blockchain ledger.

Step 16: Create a New Asset

Create a new asset named asset10.

```
peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
-c
'{"function":"CreateAsset","Args":["asset10","green","10","Alice","500"]
}'
```

The asset contains the following details:

- **Asset ID:** asset10
- **Color:** green
- **Size:** 10
- **Owner:** Alice
- **Appraised Value:**

500 **Step 17: Read the**

Asset Query the details

of asset10.

```
peer chaincode query -C mychannel -n basic \
  -c '{"function":"ReadAsset","Args":["asset10"]}'
```

Expected Output:

{
"ID": "asset10",
"Color":
"green", "Size":
10, "Owner":
"Alice",
"AppraisedValue": 500
}

Step 18: Transfer the Asset

Transfer ownership of asset10 from Alice to
Bob. peer chaincode invoke -o
localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
-c '{"function":"TransferAsset","Args":["asset10","Bob"]}'

The ownership of asset10 is changed from **Alice** to
Bob.

Step 19: Verify the Updated Asset

Query the asset again to verify the new
owner. peer chaincode query -C
mychannel -n basic \
-c '{"function":"ReadAsset","Args":["asset10"]}'

Expected Output:

{
"ID": "asset10",
"Color": "green",
"Size": 10,
"Owner": "Bob",

"AppraisedValue": 500

}

Step 20: Stop the Fabric Network

After completing the experiment, stop the Fabric network.

./network.sh down

This stops and removes the Fabric containers and test network resources.

FLOW OF THE EXPERIMENT

Update Ubuntu

|

▼

Install Required Packages

|

▼

Install Docker

|

▼

Install Node.js & npm

|

▼

Install Go

|

▼

Download Hyperledger Fabric

|

▼

Configure Environment Variables

|

▼

Start Fabric Test Network

|

▼

Create Application Channel


Deploy JavaScript Chaincode
Package Chaincode
Install Chaincode
Approve Chaincode
Commit Chaincode
Initialize Ledger
Create
Asset
Read
Asset
Transfer
Asset

Verify Updated Asset

|

▼

Stop Fabric Network

RESULT

The Hyperledger Fabric development environment was successfully configured on Ubuntu. The required Docker, Node.js, npm, Go, Git, and Fabric components were installed successfully. The Hyperledger Fabric test network was started with Certificate Authorities and the application channel was created.